

Big Data folosind Apache Spark și Delta Lake

Citirea datelor în Spark - **SparkSession**, punctul prin care comunicăm cu Spark.

În **Databricks**, organizarea datelor ca **Delta Lake** înseamnă stocarea și gestionarea datelor într-un format care combină **fișierele Parquet** cu un **jurnal de tranzacții** (transaction log). Astfel obții un *data lake* cu funcționalități de *data warehouse*: fiabilitate, consistență și performanță.

Delta Lake este un strat de stocare open-source care rulează peste data lake (ex. cloud storage sau HDFS) și adaugă:

- 1) **Tranzacții ACID** - operațiile **INSERT / UPDATE / DELETE**
- 2) **Schema enforcement & evolution** - permite **evoluția controlată** a schemei în timp.
- 3) **Versionare & Time Travel**
- 4) **Performanță** - **partitionarea** înseamnă organizarea fișierelor pe directoare, în funcție de una sau mai multe coloane. **Partitionarea** este una dintre principalele tehnici de optimizare a performanței

Organizarea datelor

- datele sunt fișiere **Parquet** într-un folder.
- în același folder există un subfolder **_delta_log/** cu fișiere JSON/Parquet ce descriu toate schimbările (jurnalul).
- Databricks citește jurnalul pentru a ști **ce fișiere sunt valide** la o versiune dată.

The screenshot shows the 'Catalog Explorer' interface in Databricks. The breadcrumb path is 'main > proiect_bigdata > trips_cleaned'. The table 'trips_cleaned' is selected, and the 'Details' tab is active. The 'About this table' section provides the following information:

Created at	Nov 07, 2025, 08:29 PM
Created by	mironela.pirnaeu@prof.utm.ro
Delta runtime properties kv pairs	(empty)
Storage location	s3://utm-dbx-platform-metastore-storage/metastore/1c85d2e6-5f71-4795-a58d-d0dc9e606e350-0fb8-4430-8cd3-819c80ef6918
Table ID	e606e350-0fb8-4430-8cd3-819c80ef6918
Type	MANAGED

The 'Properties' section shows the following Delta Lake configuration:

Properties	delta: - enableDeletionVectors: "true" - feature.appendOnly: "supported" - feature.deletionVectors: "supported" - feature.invariants: "supported" - lastCommitTimestamp: "1762540152000" - lastUpdateVersion: "0" - minReaderVersion: "3"
------------	--

```
► Run suggested ❶ Last execution failed 1: Cell 1

dbutils.fs.ls(
  "s3://utm-dbx-platform-metastore-storage/metastore/1c85d2e6-5f71-4795-a58d-d0dc96124bed/
  tables/4275ab86-835d-4644-b05e-074eacca665d"
)
1 # Instead of accessing S3 directly, use DESCRIBE DETAIL to see table metadata
2 spark.sql("DESCRIBE DETAIL main.proiect_bigdata.trips_cleaned").display()
> See performance (1) ❌ 1

❶ > ExecutionError: (org.apache.spark.SparkSecurityException) [INSUFFICIENT_PERMISSIONS] Insufficient privileges:
User does not have permission SELECT on any file. SQLSTATE: 42501
```

```
▶ Just now (2s) 2: Untitled Python
# Or read the table directly using Spark
df = spark.read.table("main.proiect_bigdata.trips_cleaned")
print(f"Number of files: {len(df.inputFiles())}")
df.show(5)
> See performance (1)

> df: pyspark.sql.connect.dataframe.DataFrame = [tpep_pickup_datetime: timestamp, tpep_dropoff_datetime: timestamp ...
4 more fields]
Number of files: 1
```

- Se citește **tabela trips_cleaned** direct din metastore ca Delta Lake.
- Databricks consultă intern **_delta_log** pentru a afla ce fișiere Parquet sunt valide și construiește DataFrame-ul.
- Metoda **inputFiles()** arată câte fișiere de date sunt folosite la citire.
- Rezultatul „Number of files: 1” indică faptul că tabela este stocată într-un singur fișier Parquet (date puține sau scriere într-un singur task).
- Folderul **_delta_log** nu apare deoarece conține doar metadate și este gestionat automat de Databricks.

SparkSession este „poarta” prin care aplicația Python poate lansa operații distribuite. Citirea tabelului `store_sales` nu înseamnă că datele sunt încărcate integral în memorie locală. Spark lucrează **lazy** (incent): doar definește planul de execuție, doar când apelăm **show()**, Spark se declanșează citirea efectivă.

Datele sunt prea mari pentru a fi procesate pe un singur calculator, Spark le împarte automat pe mai multe noduri.

Exemplu

```
from pyspark.sql import SparkSession
# Inițializăm sesiunea Spark
spark = SparkSession.builder.getOrCreate()
```

```
# Citim datele brute despre cursele de taxi
df_raw = spark.read.table("samples.nyctaxi.trips")
# Afișăm primele 5 înregistrări
df_raw.show(5)
```

Explorarea structurii și selecția coloanelor

`printSchema()` ne arată tipurile de date și ne ajută să evităm erori ulterioare.

```
# Afișăm schema tabelului
df_raw.printSchema()
```

```
# Selectăm doar coloanele relevante pentru analiză
```

```
df_selected = df_raw.select(
    "tpep_pickup_datetime",
    "tpep_dropoff_datetime",
    "trip_distance",
    "fare_amount",
    "pickup_zip",
    "dropoff_zip"
)
df_selected.show(5)
```

<https://www.databricks.com/blog/crossing-bridges-reporting-nyc-taxi-data-rstudio-and-databricks>

Câmpurile selectate au rolul de a descrie **când, unde, cât și cu ce cost** are loc o cursă de taxi.

- **tpep_pickup_datetime** – momentul începerii cursei (data și ora), poate fi folosit pentru analize în timp (ore de vârf, zile, luni).
- **tpep_dropoff_datetime** – momentul încheierii cursei, permite calculul **duratei cursei**.
- **trip_distance** – distanța parcursă (de obicei în mile), utilă pentru analiza curselor scurte vs. lungi.
- **fare_amount** – costul de bază al cursei (fără taxe și bacșiș).
- **pickup_zip** – zona (cod poștal) de unde a început cursa, pt analiza cererii pe zone.
- **dropoff_zip** – zona unde s-a terminat cursa, pt. analiza fluxurilor între zone.

Curățarea datelor (Data Cleaning)

Pentru a realiza acest proces de curățare, este utilizată **funcția col**, importată din modulul `pyspark.sql.functions`. Această funcție permite referirea explicită la coloanele **unui DataFrame Spark** și aplicarea de operații logice asupra acestora. Utilizarea **funcției col** este o practică standard în Apache Spark, deoarece oferă claritate și flexibilitate în definirea expresiilor de filtrare, mai ales atunci când sunt implicate mai multe condiții.

Operația de curățare propriu-zisă este realizată prin intermediul metodei **filter()**, aplicată asupra `DataFrame`-ului care conține coloanele relevante pentru analiză. Prin această metodă sunt eliminate acele înregistrări pentru care distanța cursei sau tariful perceput sunt mai mici sau egale cu zero. Astfel de valori pot apărea ca urmare a unor erori de măsurare, a problemelor tehnice sau a introducerii incorecte a datelor și nu reflectă situații reale de transport.

Condiția de filtrare este construită prin combinarea a două expresii logice, utilizând operatorul **AND (&)**, ceea ce impune ca ambele condiții să fie îndeplinite simultan pentru ca o înregistrare să fie păstrată. Această abordare asigură că setul de date rezultat conține exclusiv curse valide, caracterizate printr-o distanță pozitivă și un tarif corespunzător.

Rezultatul acestui proces este obținerea unui nou DataFrame, **denumit df_clean**, care conține date curate și coerente, pregătite pentru etapele ulterioare de analiză. În Apache Spark, DataFrame-urile sunt imutabile, ceea ce înseamnă că operațiile de transformare nu modifică datele originale, ci generează un nou set de date. Această caracteristică contribuie la siguranța și predictibilitatea procesării distribuite.

Pentru verificarea vizuală a rezultatului obținut, este utilizată metoda `show(5)`, care afișează primele cinci înregistrări din DataFrame-ul curățat. Această operație declanșează execuția efectivă a planului de procesare definit anterior, permițând confirmarea faptului că filtrarea a fost aplicată corect.

```
from pyspark.sql.functions import col
# Eliminăm cursele invalide
df_clean = df_selected.filter(
    (col("trip_distance") > 0) &
    (col("fare_amount") > 0)
)
```

```
df_clean.show(5)
```

Salvarea datelor curate în Delta Lake

```
df_clean.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("main.proiect_bigdata.trips_cleaned")
```

După curățare, datele sunt salvate într-un format Delta Lake, optimizat pentru Big Data.

Citirea datelor procesate

```
df_clean = spark.read.table("main.proiect_bigdata.trips_cleaned")
```

```
df_clean.printSchema()
```

```
df_clean.show(5)
```

Acest pas demonstrează **separarea etapelor**: date brute, date curate, analiză.

Calculul duratei medii a curselor

Calculăm durata fiecărei curse folosind timestamp-uri, apoi obținem media.

Spark împarte datele pe noduri, calculează parțial rezultatele și le combină automat.

Agregările sunt distribuite, nu secvențiale.

`unix_timestamp`, `avg` și `round`, aceasta din urmă fiind redenumită sub aliasul `spark_round` pentru a evita conflictele cu funcția omonimă din Python.

Funcția **`unix_timestamp`** este utilizată pentru a transforma valorile de tip dată și timp în valori numerice, exprimate în secunde de la un moment de referință standard (1 ianuarie 1970).

Funcția este aplicată asupra coloanelor care conțin momentul preluării și momentul finalizării cursei. **Conversia în valori numerice este necesară deoarece operațiile aritmetice directe nu pot fi aplicate pe date de tip timestamp.** Prin scăderea celor două valori obținute se determină durata fiecărei curse în secunde.

Rezultatul acestei diferențe este apoi împărțit la 60, pentru a converti durata din secunde în minute, unitate de măsură mai intuitivă și mai ușor de interpretat în analiza transportului urban. Această operație este aplicată implicit tuturor înregistrărilor din DataFrame, Spark ocupându-se de distribuirea calculelor pe mai multe noduri de procesare.

Funcția avg este utilizată pentru a calcula media aritmetică a duratelor obținute. Aceasta este o funcție de agregare, ceea ce înseamnă că Spark calculează mai întâi medii parțiale pe fiecare partiție de date, iar apoi le combină pentru a obține rezultatul final. Acest mecanism face posibilă analiza eficientă a unor volume foarte mari de date, fără a fi necesară încărcarea lor într-un singur nod.

Pentru a îmbunătăți lizibilitatea rezultatului final, este utilizată funcția round, redenumită în cod spark_round. Aceasta rotunjește valoarea numerică rezultată la două zecimale, făcând informația mai ușor de interpretat și mai potrivită pentru prezentare sau raportare. Utilizarea aliasului este o bună practică, deoarece evită confuziile cu funcțiile standard ale limbajului Python.

Întregul calcul este realizat în cadrul metodei select, care creează un nou DataFrame ce conține o singură coloană, reprezentând durata medie a curselor. Prin utilizarea metodei alias, această coloană este denumită sugestiv durata_medie_minute, ceea ce contribuie la claritatea semantică a rezultatului.

```
from pyspark.sql.functions import unix_timestamp, avg, round as spark_round
df_durata = df_clean.select(
    spark_round(
        avg(
            (unix_timestamp("tpep_dropoff_datetime") -
             unix_timestamp("tpep_pickup_datetime")) / 60
        ), 2
    ).alias("durata_medie_minute")
)
```

```
df_durata.show()
```

Analiza duratei în funcție de oră

Grupăm cursele în funcție de ora din zi pentru a observa tipare: trafic mai mare dimineața și seara, durate diferite în funcție de oră.

```
from pyspark.sql.functions import hour
```

```
df_ore = df_clean.groupBy(
    hour("tpep_pickup_datetime").alias("ora")
).agg(
    spark_round(
        avg(
            (unix_timestamp("tpep_dropoff_datetime") -
             unix_timestamp("tpep_pickup_datetime")) / 60
        ), 2
    ).alias("durata_medie_minute")
).orderBy("ora")
```

```
df_ore.show()
```

import hour - extrage componenta „oră” dintr-o valoare de tip timestamp. Aplicată asupra coloanei tpep_pickup_datetime, funcția hour transformă un moment complet de timp (dată și oră) într-o valoare numerică între 0 și 23, corespunzătoare orei din zi în care a început cursa. Această transformare este esențială pentru gruparea curselor pe intervale orare.

Funcția agg, prescurtare de la *aggregate*, este utilizată în Apache Spark pentru realizarea operațiilor de agregare asupra datelor. Aceasta are rolul de a sintetiza un volum mare de înregistrări într-un set redus de valori reprezentative, cum ar fi media, suma, numărul de apariții

sau valorile minime și maxime. În contextul procesării Big Data, funcția agg este esențială deoarece permite extragerea informațiilor relevante din seturi de date de mari dimensiuni, care nu pot fi analizate eficient la nivel de rând individual.

Funcția agg este utilizată, de regulă, în combinație cu metoda groupBy. În urma operației groupBy, datele sunt împărțite în grupuri distincte pe baza unei caracteristici comune, cum ar fi ora, ziua sau o anumită categorie. Ulterior, **funcția agg este aplicată fiecărui grup pentru a calcula valori sintetice care descriu comportamentul general al acelui grup.** Astfel, un număr mare de înregistrări este transformat într-o singură valoare agregată pentru fiecare grup.

În cadrul analizei curselor de taxi, funcția agg este utilizată pentru a calcula durata medie a curselor pentru fiecare oră a zilei. Pentru fiecare grup orar, Spark calculează mai întâi durata fiecărei curse, apoi determină media acestor durate. Acest proces este realizat în mod distribuit, fiecare nod de procesare calculând rezultate parțiale, care sunt ulterior combinate pentru a obține rezultatul final.

Un aspect important al funcției agg este faptul că aceasta reduce semnificativ volumul de date rezultat. În loc să fie păstrate toate înregistrările individuale, rezultatul agregării conține un număr redus de rânduri, fiecare reprezentând un grup distinct. Această reducere a volumului de date facilitează interpretarea rezultatelor și permite utilizarea acestora în rapoarte, grafice sau dashboard-uri.

Analiza tarifului în funcție de distanță

```
df_tarif = df_clean.filter(
    (col("trip_distance") > 0) & (col("trip_distance") <= 30)
).groupBy("trip_distance").agg(
    spark_round(avg("fare_amount"), 2).alias("tarif_mediu")
).orderBy("trip_distance")
```

Funcția toPandas() în Apache Spark

Funcția toPandas() este o metodă aparținând clasei DataFrame din biblioteca PySpark. Aceasta este definită în modulul pyspark.sql.dataframe și este disponibilă pentru orice obiect de tip DataFrame Spark.

toPandas() nu trebuie importată explicit, poate fi apelată direct asupra unui DataFrame deja existent.

Rolul funcției toPandas() este de a converti un DataFrame Spark, care este procesat distribuit pe un cluster, într-un DataFrame Pandas, care este procesat local, în memoria unui singur nod. Această conversie presupune colectarea tuturor datelor din DataFrame-ul Spark și transferarea lor în memoria locală a aplicației Python.

Din punct de vedere tehnic, toPandas() funcționează ca o operație de tip *action* în Spark. Până la apelul acestei metode, Spark construiește doar un plan logic de execuție. În momentul în care toPandas() este apelată, Spark declanșează execuția efectivă a planului, colectează datele de pe toate nodurile clusterului și le aduce într-o structură Pandas.

Utilizarea funcției toPandas() este justificată în special în etapele finale ale unui pipeline Big Data, atunci când datele au fost deja agregate și volumul lor este suficient de mic pentru a putea fi gestionat în memoria locală. Aceasta este o practică frecvent întâlnită în procesul de vizualizare a datelor sau în analiza exploratorie, unde bibliotecile Python precum Pandas, Matplotlib sau Plotly sunt mai potrivite decât Spark.

toPandas() nu trebuie utilizată pe seturi de date foarte mari, deoarece aceasta poate conduce la depășirea memoriei disponibile pe nodul local. Din acest motiv, conversia este recomandată doar după aplicarea operațiilor de filtrare și agregare, astfel încât volumul de date rezultat să fie redus.

Machine Learning pe date Big Data folosind Apache Spark in Databricks.

```
# 1. Importuri necesare
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
# 2. Citim datele de ML din tabelul Delta
df = spark.table("`utm-master-an2-sd&ia-dl`.mironela.features_ml_table")
# 3. Construim vectorul de feature-uri
assembler = VectorAssembler(
    inputCols=["num_transactions"],
    outputCol="features"
)
df_ml = assembler.transform(df).select("features", "total_spent")
# 4. Impartim datele in train si test
train_df, test_df = df_ml.randomSplit([0.8, 0.2], seed=42)
# 5. Definim modelul de regresie liniara
lr = LinearRegression(
    featuresCol="features",
    labelCol="total_spent"
)
# 6. Antrenam modelul
model = lr.fit(train_df)
# 7. Aplicam modelul pe datele de test
predictions = model.transform(test_df)
# 8. Evaluam modelul folosind RMSE
evaluator = RegressionEvaluator(
    labelCol="total_spent",
    predictionCol="prediction",
    metricName="rmse"
)
rmse = evaluator.evaluate(predictions)
# 9. Afisam rezultate
print("RMSE:", rmse)
predictions.select("features", "total_spent", "prediction").show(10, truncate=False)
```

1. Citirea datelor

Citirea tabelului `features_ml_table` aduce in Spark un set de date **mic, stabil si agregat**, unde fiecare rand reprezinta un client intr-un an. Acest pas marcheaza **sfarsitul procesarii Big Data** si inceputul procesarii de tip Machine Learning.

Este foarte important ca ML sa nu lucreze direct pe date brute, ci pe date deja agregate si curate.

2. VectorAssembler – transformarea coloanelor in feature-uri

Spark ML nu lucreaza cu coloane separate, ci cu un **vector numeric unic** numit `features`. VectorAssembler transforma coloana `num_transactions` intr-un vector de forma:

```
[num_transactions]
```

Acesta este pasul clasic de **feature engineering** pentru ML.

3. Selectia label-ului

Coloana total_spent este aleasa ca **label**, adica valoarea pe care vrem sa o prezicem. In acest caz, vrem sa invatam relatia dintre: cate tranzactii face un client, cat cheltuiește acel client

4. Impartirea in train si test

Datele sunt impartite in: 80% date de antrenare, 20% date de test pentru a evalua corect modelul si pentru a evita supra-invatarea (overfitting).

5. Alegerea modelului – regresie liniara

Regresia liniara este folosita deoarece: problema este una numerica, este simpla si usor de explicat, permite interpretare directa

Modelul invata o relatie de tipul:

$$\text{total_spent} = a * \text{num_transactions} + b$$

6. Antrenarea modelului

Sparkparcurge datele de antrenare, calculeaza coeficientii modelului, construiește un obiect model reutilizabil

Acesta este singurul moment in care are loc **Machine Learning propriu-zis**.

7. Predictii

Modelul este aplicat pe datele de test. Pentru fiecare rand: total_spent este valoarea reala, prediction este valoarea estimata de model. Acest pas permite inspectarea vizuala a calitatii predictiilor.

8. Evaluare – RMSE

RMSE (Root Mean Squared Error) masoara: cat de mare este eroarea medie a predictiei, in aceleasi unitati ca label-ul (total_spent). Nu cautam un scor perfect, ci **intelegerea fluxului complet**.

9. Mesajul-cheie al lectiei

Machine Learning-ul a fost **partea cea mai simpla** din tot procesul. Cea mai mare parte a muncii a fost: integrarea datelor, partitionarea, reducerea volumului, feature engineering.

ML nu esueaza din cauza algoritmilor, ci din cauza datelor pregatite. Datele au fost citite, transformate, împărțite, modelul a fost antrenat, predicțiile au fost generate

Pipeline-ul ML a rulat complet.


```

|
└─ (4) Spark Jobs
  > df: pyspark.sql.dataframe.DataFrame = [ss_customer_sk: integer, d_year: integer, ...]
  > df_ml: pyspark.sql.dataframe.DataFrame
  > predictions: pyspark.sql.dataframe.DataFrame
  > test_df: pyspark.sql.dataframe.DataFrame
  > train_df: pyspark.sql.dataframe.DataFrame

RMSE: 7890.025138515756

+-----+-----+-----+
|features|total_spent|prediction|
+-----+-----+-----+
|[6.0]|6135.65|15487.551301414042|
|[6.0]|12874.89|15487.551301414042|
|[7.0]|5795.19|16232.819283344492|
|[7.0]|10513.66|16232.819283344492|
|[7.0]|13713.85|16232.819283344492|
|[7.0]|20009.60|16232.819283344492|
|[7.0]|31035.87|16232.819283344492|
|[8.0]|8069.17|16978.087265274942|
|[8.0]|11569.96|16978.087265274942|
|[8.0]|11915.24|16978.087265274942|
+-----+-----+-----+
only showing top 10 rows

```

Aplicatii

DESCRIBE TABLE samples.tpcds_sf1000.store_sales;

DESCRIBE DETAIL samples.tpcds_sf1000.store_sales;

CREATE OR REPLACE TABLE `utm-master-an2-sd&ia-dl`.mironela.store_sales_small
USING DELTA

AS

SELECT

ss.*,

d.d_year

FROM samples.tpcds_sf1000.store_sales ss

JOIN samples.tpcds_sf1000.date_dim d

ON ss.ss_sold_date_sk = d.d_date_sk

WHERE d.d_year = 2001

LIMIT 5000;

SELECT COUNT(*)

FROM `utm-master-an2-sd&ia-dl`.mironela.store_sales_small;

DESCRIBE DETAIL `utm-master-an2-sd&ia-dl`.mironela.store_sales_small;

CREATE OR REPLACE TABLE `utm-master-an2-sd&ia-dl`.mironela.store_sales_small_part
USING DELTA

PARTITIONED BY (d_year)

AS

SELECT *

```
FROM `utm-master-an2-sd&ia-dl`.mironela.store_sales_small;
```

Partitionarea înseamnă că datele nu mai sunt stocate ca o masă unică de fișiere, ci sunt **separate fizic pe disk** în funcție de o coloană `d_year`. Astfel, atunci când rulăm un query cu un filtru pe acea coloană, motorul Big Data **nu mai scanează tot tabelul**, ci **doar partițiile relevante**. Acest mecanism se numește *partition pruning*.

PARTITIONED BY (...) înseamnă **partitionare fizică a datelor pe disc**, pe directoare.

partitionare === performanța.

```
DESCRIBE DETAIL `utm-master-an2-sd&ia-dl`.mironela.store_sales_small_part;
```

```
EXPLAIN
```

```
SELECT SUM(ss_net_paid)
```

```
FROM `utm-master-an2-sd&ia-dl`.mironela.store_sales_small_part
```

```
WHERE d_year = 2001;
```

```
CREATE OR REPLACE TEMP VIEW features_ml AS
```

```
SELECT
```

```
  ss_customer_sk,
```

```
  d_year,
```

```
  SUM(ss_net_paid) AS total_spent,
```

```
  COUNT(*) AS num_transactions
```

```
FROM `utm-master-an2-sd&ia-dl`.mironela.store_sales_small_part
```

```
GROUP BY ss_customer_sk, d_year;
```

```
CREATE OR REPLACE TABLE `utm-master-an2-sd&ia-dl`.mironela.features_ml_table
```

```
USING DELTA
```

```
AS
```

```
SELECT *
```

```
FROM features_ml;
```

```
CREATE OR REPLACE TEMP VIEW features_ml AS
```

```
SELECT
```

```
  ss_customer_sk,
```

```
  d_year,
```

```
  SUM(ss_net_paid) AS total_spent,
```

```
  COUNT(*) AS num_transactions
```

```
FROM `utm-master-an2-sd&ia-dl`.mironela.store_sales_small_part
```

```
GROUP BY ss_customer_sk, d_year;
```

```
SELECT *
```

```
FROM `utm-master-an2-sd&ia-dl`.mironela.features_ml_table;
```
